

Thimble: A 48M-Parameter Tool-Calling Model from Co-Designing the Tokenizer, Loss, and Decoder

— with eleven negative results —

Nikshep S. (nikshepsvn)

nikshepsvn@gmail.com

<https://github.com/nikshepsvn/thimble>

August 2026

Abstract

Turning a natural-language request into calls against a fixed, typed API is a smaller problem than the general-purpose models usually pointed at it. We treat it as a translation layer rather than a language model: Thimble is a 48.12M-parameter model that reads a catalog of JSON function schemas and a request, and emits the calls to make — or an empty list — through a grammar-constrained decoder. The tokenizer, the training loss, and the decoder are designed together around the same five decision points, rather than a grammar being bolted onto a free-text model at inference time. Two measurements suggest the co-design is load-bearing: constrained and unconstrained decoding agree on 150 of 150 evaluation rows (there is no “projection tax” to recover), and down-weighting the grammar-forced tokens in the loss — on the theory that the decoder supplies them anyway — costs 12 points in a controlled twin run, a small-scale counterexample to a null result reported at larger scale. On catalogs represented in training, Thimble scores 86.3 on Mobile Actions and 52.5 on DroidCall (ordered strict exact match) against a published 45M-parameter yardstick’s 63.7 and 17.0; on unseen catalogs it drops to rough parity, and we show the failure is function selection, not argument extraction. Alongside the model we publish eleven negative results, each killed by a controlled measurement rather than an argument, and a single-file dependency-free C inference engine whose fp32 output is byte-identical to the reference stack on every row checked. Total training cost across all cycles was about \$260. Weights, training pipeline, evaluation harness, and the full chronological experimental record are released under MIT.

1 Introduction

Most systems that need tool calling obtain it from a general-purpose model: either a frontier model prompted with function schemas, or a sub-billion-parameter model fine-tuned for the task [4, 7, 19]. Both inherit machinery the task does not obviously need — open-vocabulary generation, world knowledge, conversational ability — and both keep a failure mode the task cannot tolerate: nothing structurally prevents malformed JSON, invented parameter names, or calls to undeclared functions. Constrained decoding [2, 5, 9, 14] removes those failure modes at inference time, but the standard arrangement bolts a grammar onto a model that was trained for free text and then manages the mismatch: token healing at constraint boundaries, logit masking over a vocabulary whose merges span structural characters, and a documented distortion of the model’s distribution [14, 16].

This report explores the opposite end of the design space. If the model will only ever emit tool calls under a grammar, the tokenizer, the training loss, and the decoder can be designed together around the decisions the grammar actually leaves open. In Thimble those decisions number five: refuse or call; which tool; include this optional argument; what value; stop or continue. Everything else — braces, quotes, commas, and every argument key — is emitted by a grammar compiled from the user’s schemas, so three properties hold on any catalog with no training or configuration: output is always well-formed

JSON; argument keys can only come from the schema, making parameter-name hallucination structurally impossible rather than unlikely; and calls to undeclared tools cannot be emitted. The whole model is 48.12M parameters, small enough that specializing a private copy to one API surface is routine.

We are explicit about what is and is not claimed. Small tool-calling-only models exist: Needle 2 [12, 13] occupies the same parameter class with far larger training resources, and our headline tables use their published numbers, on their metric, as a yardstick — not as a controlled comparison (§8). A token-level loss-weighting scheme very similar to ours appears in the Needle 2 authors’ architecture notes [13]; we do not claim it as a contribution. Training-time exploitation of output grammars has precedent [1, 6, 10]. The contributions here are narrower:

1. **A case study in tokenizer–loss–decoder co-design** for grammar-constrained generation, with the two measurements that make the case: the absence of a projection tax (§5.1), and a 12-point loss from down-weighting grammar-forced tokens (§5.2) — evidence that forced structural tokens carry call-sequencing signal even though the model is never asked to produce them. The latter is a small-scale counterexample to the null result of Gan et al. [8], who found loss-masking of constrained spans made no significant difference when fine-tuning 1B–8B models.
2. **Eleven negative results** (§6), each with the controlled measurement that killed it, including two process failures that cost real points. Several reverse conclusions that would have shipped on intuition and on literature support.
3. **An open, end-to-end artifact**: weights, the full training and synthesis pipeline, the evaluation harness, an adaptation script targeting new catalogs, the complete chronological lab record, and a single-file dependency-free C inference engine byte-identical to the reference implementation on every row checked (§7).

2 Related work

Constrained decoding. Grammar-constrained decoding for structured NLP is well developed [2, 5, 9], including analyses of the distortion constraints induce on a free-text model’s distribution [14] and draft-then-constrain schemes that recover accuracy lost to that distortion [16]. All of this work treats the model as fixed. Thimble differs in premise: the model is trained from scratch for exactly the constrained task, and §5.1 measures the resulting distortion at approximately zero, which is why draft-then-constrain had nothing to recover here.

Structure-aware training. GraFT [6] applies grammar masking during training for natural-language-to-temporal-logic translation and argues theoretically that masking invalid tokens accelerates convergence. SLGM [10] combines format-aware losses with format-constrained decoding for structured prediction with <1B models. Ailem et al. [1] bias an NMT loss toward terminology-constraint tokens. Closest to our loss question, Gan et al. [8] explicitly tested masking the loss on grammar-constrained spans when fine-tuning LLMs for coreference and found no significant effect. Our controlled twin run finds the opposite at 48M scale trained from scratch (§5.2); we conjecture the difference is that a from-scratch specialist must learn sequencing structure that a pretrained LLM already has.

Small tool-calling models. TinyAgent [7] fine-tunes 1.1B/7B models for edge function calling; Octopus v2 [4] trains a 2B on-device model with functional tokens; xLAM [19] releases a family of action models down to 1B. In the sub-100M class, Needle 2 [12, 13] is the published reference point: a 45M attention-only architecture trained on 153B tokens with quantization-aware training and a production C++ engine. Thimble reaches its parameter class from a different direction — a boring dense trunk, ~1B unique training tokens, no pretraining phase, and the co-design thesis above — which makes the comparison informative even where it is not controlled.

Retrieval and selection. The decoder refreshes its tool-candidate set between emitted calls, conditioning on the partial plan, following the dynamic tool retrieval argument of Patel et al. [15]. Tool-name selection is treated as a readout over candidate name spans in the prompt rather than open-vocabulary generation, following the diagnosis of Chen [3] that mis-selection is a readout failure; §5 confirms their prediction that the readout helps only off-distribution.

3 Design

3.1 The five decisions

A canonical call array over a catalog leaves the model exactly five kinds of choice: (1) refuse or call, after the opening bracket; (2) which tool, among retrieved candidates; (3) include this optional argument, for each optional key in canonical order; (4) what value, for each included argument; (5) stop or continue, after each call. The grammar, compiled from the catalog’s JSON schemas, emits everything else. Argument keys are iterated from the schema, so there is no step at which an invented key could be produced.

3.2 Tokenizer

The tokenizer is built for the grammar rather than for compression. JSON structural characters (`{ } [ ] , : "`) are singleton tokens excluded from merging, so no token ever spans a value/structure boundary: structure can be force-fed exactly, with no token healing and no ambiguity about where a constraint lands. Digits are also singletons and never merge — BPE over our corpus had produced 2021 as one token but 38.0 as three, and copy errors like $300 \rightarrow 30000$ traced directly to inconsistent numeric segmentation. The 16,384-entry vocabulary was verified by a fragmentation gate (word-value fragmentation $2.72 \rightarrow 2.34$ tokens/word, digit values lengthening by design). The rebuild shipped inside a bundle (with 350k added corpus rows and mix reweighting) that moved tool-name-sequence accuracy from 80.4% to 91.5%; the tokenizer’s isolated share was not measured, which we note rather than obscure.

3.3 Loss

Cross-entropy is weighted by decision type: structure $1\times$, argument keys $1.5\times$, tool names $2\times$, argument values $4\times$, and the stop/continue token $6\times$, matched to the measured error distribution. A closely related scheme (structure $1\times$, keys $1.5\times$, names $2\times$, values $4\times$) appears independently in the Needle 2 architecture notes [13]; the stop-decision term is ours, motivated by the multi-call collapse diagnosed in §4. The important finding about this loss is negative-space: the grammar-forced tokens, 46.4% of the weighted loss budget, must *not* be down-weighted (§5.2).

3.4 Decoder

The decoder consults the model only at the five decision points. Structural branches are decided by first-token logits, which is exact because options begin with distinct singleton tokens. Literal choices (tool names, enum values, optional-inclusion) are scored by length-normalized teacher-forced log-probability with exact snapshot/rollback of the KV cache. String values are generated freely until the closing-quote singleton, with special tokens banned and periodic k -gram repetition ($k \leq 4$, three consecutive periods) treated as a stop; an earlier blanket repeated-trigram ban mangled long natural values and cost 51 points on Mobile Actions before being diagnosed. Number values are generated under a numeric-character constraint with a structural-preference stop. Between calls the candidate set is re-retrieved conditioned on the partial plan [15]. Tool-name choice ensembles a trained bilinear readout over candidate name spans [3] with the LM’s teacher-forced score; a lexical prior over the catalog is blended in only in proportion to how sharply it discriminates *and* how uncertain the model is, which matters for refusal calibration on single-tool catalogs.

suite	Thimble v6	Needle 2 (45M)
<i>catalog represented in training (eval rows firewalled out)</i>		
Mobile Actions (961)	86.3	63.7
Mobile Actions, 2+-call rows	73.5	48.4
DroidCall (200)	52.5	17.0
Seal-Tools in-domain (700)	33.1	32.6
well-formed JSON	100.0	93.4
<i>catalog never seen</i>		
Seal-Tools out-of-domain (654)	28.1	28.7
BFCL v4 single-turn (3,641)	23.5	42.6

Table 1: Ordered strict exact match: a row passes only if function names, call order, and every argument value match. The right column is a yardstick, not a controlled comparison: Needle 2’s published numbers on their metric, unmodified, measured through their 2-bit production engine with a 256-token window, against our fp32 checkpoint (asymmetries in both directions; §8). The Seal-in margin of +0.5 on 700 rows is within sampling noise and is stated as such. DroidCall carries a permanent caveat: the benchmark’s own split script is unseeded, so our 200 rows are a seeded split from the same pool, firewalled from training — same methodology, different rows.

4 Training

Data. The corpus mixes public tool-calling sets (xLAM, ToolACE, Seal-Tools and DroidCall train splits, dria-pythonic, hermes-fc, Mobile Actions train) with teacher-synthesized rows validated against parameter types and an evidence rule: every scalar argument value must be evidenced in the query, because wrong argument values were the largest error source and rows that reward inventing values are worse than no rows. Contamination against all evaluation queries was verified by exact normalized overlap and an 8-gram firewall (one coincidental collision found and removed); the firewall is also applied to our own synthesized data.

Error-driven iteration. Row accuracy factors as $P(\text{name sequence}) \times p^n$ for per-call argument accuracy p . Each version measured which factor was binding and attacked only that. The v6 round is representative: of 193 failing calls on the previous checkpoint, 66 added exactly one unmentioned optional argument, ~35 bound the wrong entity, ~30 missed canonical date forms, and ~29 were unwinnable gold-typing noise (§6). Three matching synthesis modes (74,250 accepted rows, §56) were annealed into the learning-rate decay phase of a continued run rather than retrained from scratch: the same corrective corpus scored 28.4 fed from step zero and **33.1** annealed into decay, with a mid-run probe at constant learning rate attributing +3.3 points to the corpus itself. Corrective data dilutes when it competes with the whole corpus and concentrates when it arrives late; timing is a hyperparameter of the dataset.

Trunk. The architecture is deliberately standard: $d=448$, 20 layers, GQA 8/4, SwiGLU 2.0 $\times$, QK-norm, sandwich RMSNorm, tied embeddings, 768-token context, Muon on 2D weights with AdamW elsewhere. A controlled study by the Needle 2 authors [13] finds architecture choices at this scale worth hundredths of a nat at matched parameters; our experience matched — every real gain came from tokenizer, loss, or data changes — so the trunk is not where any claim lives.

5 Results

Table 1 gives the headline numbers on Mobile Actions, DroidCall, Seal-Tools [17], and BFCL v4 single-turn [18]. The cleanest control in the project is *within* one suite: Seal-Tools in-domain versus out-of-domain differ only in whether the catalog was represented in training, and row accuracy (33.1 vs 28.1) tracks tool-name-sequence accuracy (88.0 vs 79.0) exactly. The model does not fail to extract arguments on

unfamiliar catalogs; it fails to pick the right function. This localizes the generalization gap in a component — selection — that per-catalog adaptation is designed to fix, and explains BFCL, where every deficit category traces to selection over unfamiliar schema dialects (`simple_python` 29.3 vs `simple_java` 14.0 and `simple_javascript` 8.0 for v6).

5.1 There is no projection tax

suite (150 rows each)	free generation	grammar-constrained	unparseable (free)
Mobile Actions	78.7	78.7	0
Seal-Tools in-domain	26.7	28.0	16

Table 2: The same rows decoded with and without the grammar (`scripts/draft_vs_constrained.py`). On Mobile Actions the two agree on 150 of 150 rows.

Draft-then-constrain methods [16] recover accuracy that constrained decoding costs a free-text model; the distortion they correct is real and measured in that setting [14]. On Thimble the tax measures -1.3 and 0.0 (Table 2): the model already wants what the grammar enforces, because it was never trained to want anything else. Stated plainly: the grammar buys *reliability*, not accuracy — malformed output and invented parameter names become unreachable, and the $\sim 11\%$ of Seal rows where free generation emits invalid JSON remain parseable — but “constrained decoding makes the model correct” is a different claim and not one this data supports.

5.2 Grammar-forced tokens carry call-sequencing signal

If the decoder force-feeds all structure, training the model to predict those tokens looks like waste — they are 46.4% of the weighted loss budget. In a controlled twin run (identical data, architecture, and budget; forced-token weight $0.1\times$), the down-weighted twin finished about **12 points behind everywhere**, trailing by 19 points on name-sequencing at matched mid-training steps. Structure tokens are where the model learns when a call ends and whether another follows; remove the pressure to predict them and multi-call sequencing degrades. Gan et al. [8] report that the analogous intervention (loss-masking constrained spans) made no significant difference when fine-tuning 1B–8B pretrained models — consistent with our conjecture that pretrained models already possess the sequencing structure a from-scratch specialist must learn from exactly these tokens.

5.3 The name head helps only where the model is uncertain

Ablating the span-readout name head on the v3 checkpoint: exactly zero effect on Mobile Actions (trunk name accuracy already 99.3%), $+2.6$ exact / $+20.5$ name points on Seal in-domain, and $+4.0$ exact / $+23.4$ name points ($+39\%$ relative) out-of-domain — precisely the pattern Chen [3] predicts for a readout fix. The evaluation-practice lesson was sharper than the result: an in-distribution ablation alone would have deleted the component worth the most on the suite the model was failing.

6 Eleven negative results

Each idea below was killed by a controlled measurement. Several had literature support; two reversed conclusions that would have shipped on intuition. Full details and the chronological record are in the repository’s `FINDINGS.md` and `RESULTS.md`.

1. **Word-span copying of argument values:** -30 points ($40.0 \rightarrow 10.0$ on Seal single-call, names unchanged). The supporting evidence was real — 90% of gold values appear verbatim in the query; Seal-Tools’ own error analysis attributes 70% of parameter errors to extraction; stale and incorrect value propagation is a documented failure class [11] — but “verbatim” had been measured

as *substring* containment while the mechanism enumerated *word-level* spans. Gold `manta ray` is a substring of “manta rays” but equals no word span of it. A premise can be right while its operationalization is wrong.

2. **A learned pointer/copy head:** **−16 points** (25.3 $\rightarrow$ 9.3, names unchanged). The head learns — roughly half of training samples hit both endpoints exactly — but its confidence does not correlate with its correctness, so any confidence gate admits wrong spans, and a wrong span is a guaranteed row failure where free generation often produced the right value. Gating requires calibration, not accuracy.
3. **Down-weighting grammar-forced tokens:** **−12 points** (§5.2).
4. **From-scratch retraining vs. annealing:** **−4.7 points** (28.4 vs 33.1, same corrective corpus; §4).
5. **Field-set reranking:** **−1.4 points** (fixed 3 rows, broke 13). It targeted the key-set error bucket, which training had already fixed by the time it ran. Re-diagnose before implementing; fixes have a shelf life.
6. **Beam search / RL / best-of- N : oracle-capped below target.** `pass@9` — perfect selection over nine samples, which no deployable reranker achieves — plateaued at exactly +5.0 at every sample size measured, reaching 29.3 where 32.6 was needed. For most failing rows the correct call array is not in the model’s distribution at any temperature; the gap is capability, not search.
7. **Draft-then-constrain: nothing to recover** (§5.1). Measure the quantity a technique recovers before implementing the technique.
8. **A global optional-inclusion prior: catalog-dependent, rejected before shipping.** Gold optional-inclusion rates range from 94.1% (Mobile Actions) to 33.4% (Seal-Tools) across sources; a constant that helps one catalog wrecks another. Inclusion rate is a property of the catalog, which makes it a training problem, not a decoding constant.
9. **Raising `MAX_CALLS` 4 $\rightarrow$ 6:** **0.000 change.** The cap was never binding; long rows fail for other reasons.
10. **RLOO on the annealed checkpoint: diverges at every learning rate tried** (reward 0.50 $\rightarrow$ −0.12 by update 200 at $2e-5$; healthy to update 550 then 0.92 $\rightarrow$ 0.13 at $5e-6$), while the same recipe ran stably on the conventionally-decayed sibling. Decay-phase annealing appears to leave the model in a sharper minimum that policy-gradient noise unravels; anneal and RL are in tension.
11. **Matching the benchmark’s numeric typing: not learnable.** Seal-Tools’ gold stores numerically-typed arguments as strings 13.2% of the time with 74% of parameters inconsistent; a perfect per-parameter policy ceilings at 87.8% against 86.4% for one global rule. Some of the gap between a model and a benchmark is the benchmark.

Two process failures cost real points and are reported with the same prominence. *The pre-registered champion selector picked the wrong model:* held-out loss on the general mix chose a scratch twin over the annealed one by five-thousandths of a nat, and the scratch twin scored 28.4 against 33.1 — a general-mix dev set is structurally biased against a model that just annealed *away* from that mix, so selection must use an eval-distribution-flavored (but eval-free) split whenever recipes diverge in training distribution. *A truncating default silently dropped data:* a packing default of 512 tokens dropped 47,551 rows — precisely the longest, multi-call rows the model was weakest on — while reporting success.

7 Reproducibility and the C engine

The repository contains the full pipeline — download, conversion, contamination firewall, packing, training, and evaluation — with the exact commands that rebuild the published numbers, plus `scripts/adapt.py`, which runs the same `synthesize/pack/anneal` recipe against a user-supplied catalog (the v6 cycle’s synthesis cost \$56 for 74,250 validated rows). No third-party catalog has yet been adapted end-to-end and published; the recipe is measured, the ergonomics are new.

Inference is additionally implemented as a single dependency-free C file (`cengine/thimble.c`: tokenizer, trunk, grammar decoder, name head, and retrieval, ~1,600 lines, C11 + libm). Byte-exact parity with the Python stack is the acceptance test: identical `dumps_calls` output on 100/100 Mobile Actions evaluation rows and all demo queries including the refusal. With int8 weights (per-row scales, activations quantized per matvec through NEON `sdot`), the engine is a 48MB artifact that loads in ~20ms and runs a full request — prefill plus every decision, rollout, and rollback — in 348ms on an Apple M3, against 582ms for the PyTorch stack; quantization changed 2 of 100 rows, both, coincidentally, toward gold. Comparable engine techniques (skipping the vocabulary projection on forced steps, int8 dot-product kernels, token-level validation against a reference) are described for Needle 2’s production engine [12]; ours differ in being a byte-parity port of an open reference implementation.

8 Limitations

Single-seed results. Every number here is one training run; the project’s own selector failure (28.4 vs 33.1 between sibling checkpoints) is an implicit variance estimate large enough to swallow the Seal-in margin. The two headline ablations (§5.1, §5.2) are controlled twin runs but not multi-seed. We flag which claims rest on large margins (Mobile Actions +22.6, DroidCall +35.5, the −12 and −30 ablations) and which are within noise (Seal-in +0.5). **The yardstick is not a controlled baseline.** Needle 2’s numbers are their published figures on their metric through their production engine; precision asymmetry favors us, scope asymmetry favors us (they carry no task-specific eval-catalog training data; our Mobile Actions and DroidCall train splits are in the mix, disclosed per-suite), and training-resource asymmetry favors them (~153B vs ~1B tokens). **Generalization.** Unseen catalogs drop to parity, BFCL to 23.5 vs 42.6; parallel instantiations of one schema are a distinct, worse failure (12.0; live-parallel 0.0). The model’s intended use is per-catalog specialization, not open-world calling. **Scope.** A 768-token context excludes 4.1% of BFCL rows; Java and JavaScript schema dialects are essentially absent from the corpus. **Provenance.** The project was built by one person in a few days with substantial AI assistance, for about \$260 of compute and API spend; this report was likewise drafted with AI assistance and verified by the author.

9 Conclusion

Treated as a translation layer rather than a language model, tool calling against a declared API fits in 48M parameters, and co-designing the tokenizer, loss, and decoder around the grammar — rather than constraining a free-text model after the fact — is measurably load-bearing: the constraint costs nothing at decode time precisely because the model was trained inside it, and the tokens the grammar supplies are, counterintuitively, tokens the model must still learn. The negative results are offered as the most reusable part: most of what this project learned, it learned by being wrong efficiently.

Artifacts. Code, weights, data pipeline, C engine, and the full experimental record: <https://github.com/nikshepsvn/thimble> (MIT); weights also at <https://huggingface.co/flashvenom/thimble>.

References

- [1] Melissa Ailem, Jingshu Liu, and Raheel Qader. Encouraging neural machine translation to satisfy terminology constraints. In *Findings of the Association for Computational Linguistics: ACL 2021*, 2021.
- [2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [3] Shiyang Chen. Looking is not picking: An attention-segment account of tool-selection failures in LLM agents, 2026. arXiv:2606.16364.
- [4] Wei Chen and Zhiyuan Li. Octopus v2: On-device language model for super agent, 2024. arXiv:2404.01744.
- [5] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [6] William English, Dominic Simon, Sumit Kumar Jha, and Rickard Ewetz. Grammar-forced translation of natural language to temporal logic using LLMs, 2025. arXiv:2512.16814.
- [7] Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Se Hoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Tinyagent: Function calling at the edge, 2024. arXiv:2409.00608.
- [8] Yujian Gan, Yuan Liang, Jinxia Xie, Yanni Lin, Juntao Yu, and Massimo Poesio. Low-hallucination and efficient coreference resolution with LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [9] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [10] Minhoo Lee, Junghyun Min, Yerang Kim, Woochul Lee, and Yeonsoo Lee. Structured language generation model: Loss calibration and formatted decoding for robust structure prediction and knowledge retrieval, 2024. arXiv:2402.08971.
- [11] Seiji Maekawa, Jackson Hassell, Pouya Pezeshkpour, Tom M. Mitchell, and Estevam Hruschka. Towards reliable benchmarking: A contamination free, controllable evaluation framework for multi-step LLM function calling, 2025. arXiv:2509.26553.
- [12] Henry Ndubuaku, Karen Mosoyan, Jakub Mroz, Noah Cylich, Satyajit Kumar, Parkirat Sandhu, Roman Shemet, and Justin H. Lee. Needle 2: A 45m-parameter foundation tool-calling model for tiny devices. <https://github.com/cactus-compute/needle>, 2026. Cactus Compute, Inc.
- [13] Henry Ndubuaku et al. Simple attention networks, 2026. Architecture notes: https://github.com/cactus-compute/needle/blob/main/docs/simple_attention_networks.md.
- [14] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D’Antoni. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [15] Bhrij Patel, Davide Belli, Amir Jalalirad, Maximilian Arnold, Aleksandr Ermolov, and Bence Major. Dynamic tool dependency retrieval for efficient function calling, 2025. arXiv:2512.17052.
- [16] Avinash Reddy, Thayne T. Walker, James Ide, and Amrit Singh Bedi. Draft-conditioned constrained decoding for structured generation in LLMs, 2026. arXiv:2603.03305.

- [17] Mengsong Wu, Tong Zhu, Han Han, Chuanyuan Tan, Xiang Zhang, and Wenliang Chen. Seal-tools: Self-instruct tool learning dataset for agent tuning and detailed benchmark, 2024. arXiv:2405.08355.
- [18] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>, 2024.
- [19] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025.